

CSE 144 Final Project Report

Transfer Learning Challenge

Group Members:

Sarah Reuter (CruzID: smreuter)
Catherine Mikhailova (CruzID: cmikhail)
Malco Salcedo (CruzID: mvsalced)

1. Introduction

1. Problem goal and setting. Why transfer learning is appropriate. Brief summary of your approach and main result.

The problem is that we must try to classify roughly 1000 images into 100 classes leaving 10 images per class. The best way to solve this problem is to use transfer learning on a predefined model with predefined weights. A self-made model without predefined weights will overlearn on the training set. Since it already has weights that are trained on another batch of images it will only take fine tuning to make this model able to classify images for our purposes. Since there are only roughly 863 images in the training set, we decided to start with a smaller model, because larger models are more likely to overfit. Therefore, we started with the EfficientNetB0 to set up our training/testing loop, data pipeline and ensure that the labels were mapping correctly. Then, we all broke apart and were training different models, hoping to find a model that was more suitable to the needs of this dataset. Some of the models we tried included PyTorch's ViT and RegNet before landing on a SigLIP for the highest result.

2. Dataset

1. Number of classes and dataset sizes (train/val/test). Directory structure and label mapping details (ensure labels 0–99 match folders). Preprocessing (resize, normalization) and data augmentation used.

There were 100 classes in this dataset, with a split of train/val/test: 863/216/1036. We split the dataset into an 80/20 split for training and validation because we felt like it was a good balance of having as much data as possible without making the validation set so small that it would be too noisy. For label mapping, we had to make sure that we sorted the labels in such a way that sorted the integer labels numerically instead of alphabetically because this isn't the default for ImageFolder. Moreover, it would sort the file labels `file_names={0: "0", 1: "1", 2: "10", 3: "11", 4: "12", ..., "99": 99}` instead of `file_names={0: "0", 1: "1", 2: "2", 3: "3", 4: "4", 5: "5", ..., 99: "99"}`.

The preprocessing we landed on included composing all of the following transforms so that it could be passed in while we loaded the datasets. We used v2 for transforms because the pytorch website said that it was slightly faster than the other options. The specific transforms that we used were `resize`, `RandomResizedCrop`, `RandomHorizontalFlip`, `ColorJitter`, `ToDType`, and `Normalize`. Later in the training loop, we also used `CutMix`. We then had to create an `UnlabeledDataset` class because the `ImageFolder` wouldn't work since there were no labels in the test folder. Moreover, it does the bare minimum of just getting the images, and doing the

test_tf on them, and sorting them. When we call getitem at the end, while sorting these into the csv, we return the image and the name of the image.

The directory structure we had in this process is as follows:

"contents/model.ipynb" directory in google colab. Then, we had our checkpoint in

"./checkpoints/best_transfer.ViT.pt". Lastly,

"./submissions/predictions.csv" is where we stored the csv before submission assuming that our current directory was just the root.

3. Implementation

3.1 Model

1. Pretrained backbone used (e.g., ResNet/EfficientNet/ViT) and why.

Started with EfficientNetB0, and found it peaked around 60% after a bit of fine-tuning. After adding k-Fold ensembling and moving it to EfficientNetB2 it went up to 64% accuracy on the test set. Tried a hierarchical approach with coarse grain sorting between a plane, car, food, and plants, but found after trying to train the fine-grained sets that there was even less data to work with and training wasn't happening successfully. Next, we attempted using Regnet-Y 8GF with IMAGENET1k_V1. The main reason was that it's a strong mid-sized model that balances capacity with overfitting on small datasets, but found best results with the SigLIP model.

We also concurrently tried to use ViT because we thought that implementing an attention mechanism could be helpful especially because the dataset we were provided with was so small. We eventually settled on using SigLIP. SigLIP is a multimodal image-text model that separates image and text encoders to generate representations for both modalities. SigLIP has a strong ViT backbone and uses a pairwise sigmoid loss on image-text pairs during training, which allows it to have superior performance with smaller batch sizes. Treating each image-text pair as an independent binary classification text helps to eliminate heavy memory overload.

2. Architecture changes (classifier head, pooling, dropout, etc.).

We changed the classifier head of the model to map to the classes that we needed them to. Then we also added dropout, since this is such a small dataset, dropout is a massive help to resolve the issue of overfitting. Experimenting further, we froze all backbone layers, then unfroze blocks 3 and 4 of trunk_output. We figured that unfreezing those struck the balance of leveraging pretrained data and fine tuning for our small dataset.

3. Fine-tuning strategy (frozen layers, unfreezing schedule).

We each used many different strategies per model:

VIT + LoRa: For the fine-tuning strategy, I am attempting to use Low-Rank Adaptation (LoRA). LoRA freezes the backbone but injects small trainable matrices into the attention layers. Since ViT uses multi-head self attention, I will keep the dataset (planes, cars, food, animals, etc.) all mixed together because the heads will be forced to learn more general and robust features; no single visual pattern works for everything. We found the best results on the SigLIP when we

(06.04) - Implemented LoRA → helped slightly, still trying to fine-tune ViT.

Widening rank from 4->16 since the dataset is pretty small. Rank is a hyperparameter that controls the capacity of the adapter to learn new information.

RegNet: My fine tuning strategy was to selectively unfreeze layers and combine that with layer wise learning rates. Block3 had a learning rate of $5e-5$ and block4 had a learning rate of $1e-4$. The head was at $1e-3$. My reasoning behind this was that it lets the head learn faster while gently fine tuning the rest of the features.

3.2 Training

1. Loss function and optimizer.

We used `CrossEntropyLoss()` with label smoothing at 0.1 to prevent overconfident/overfitting predictions on our dataset. We also used AdamW with weight decay as our optimizer for its decoupled weight decay set to 0.01.

2. Learning rate, scheduler, batch size, epochs, weight decay.

VIT: (06.04) - Attempting to use a learning rate scheduler to have a dynamic rate of updating the weights. Using `OneCycleLr` $\Rightarrow$ has a warmup phase for the first 20% of training, then a cosine decay for the remaining (80%). `OneCycleLR` is a per-batch scheduler.

$\Rightarrow$ helped go from 50% test accuracy to 56%

Regnet: I used a `CosineAnnealingLR` over a multitude of different epochs throughout this projects timeline. I used a layer wise learning rate as mentioned above. I found 50 epochs to be omst successful in being enough to maximize results.

My batch size was 64

3. Hardware/software environment (GPU, PyTorch version).

We attempted a mixed precision approach simply for faster gpu training times. I accomplished this using `torch.amp.autocast` and `GradScaler`.

This was accomplished on Google Colab, with a T4 GPU and of course Pytorch2.11.0+cu128

4 Experiments

1. Baseline setup.

Our baseline was EfficientNetB0 with the full backbone frozen, a new linear classification head, standard Resize + Normalize preprocessing (no augmentation), and a flat learning rate of $1e-3$ trained for 20 epochs. This baseline achieved approximately 50-55% validation accuracy and served as a reference point for all subsequent improvements.

2. Hyperparameter tuning plan and validation method.

RegNet: My hyperparameter tuning plan started by unfreezing the single block4 using AdaGrad. Through iteration I found that AdamW was a much better optimizer along with differential learning rates, which led me to unfreeze block3. I increased regularization through dropout starting from 0.3 to 0.5 and weight decay from $1e-4 \rightarrow 5e-3$. I experimented with `CosineAnnealingWarmRestarts` over 80 epochs and tested CutMix. That wasn't successful so it was removed. My final configuration was differential LR's ($5e-5$, $1e-4$, $1e-3$), weight decay at $53-3$, dropout 0.5, label smoothing 0.1 and warm restarts over 80 epochs.

My validation method involved having a fixed 85/15 training/val split with a seed set to 42 for reproducibility. Seed 42 was used for all our models to ensure reproducible results.

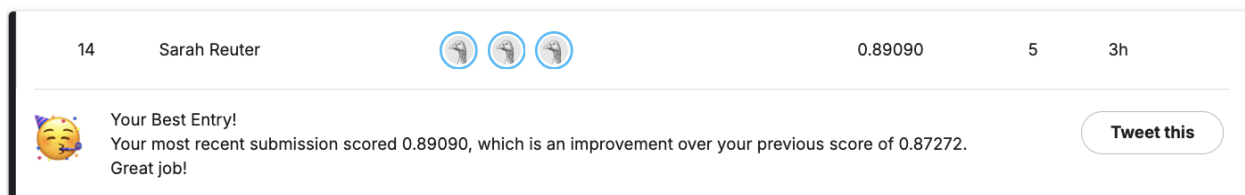
3. Ablations (augmentations, model size, freezing strategy, LR, etc.).

Augmentation (RandomResizedCrop, ColorJitter, CutMix), consistently reduced overfitting on EfficientNet models. Label smoothing provided a small but consistent improvement over no smoothing. Using LoRA helped on the small dataset where adapters needed more capacity.

5 Results

1. Training/validation accuracy (and loss) and any key curves/plots (optional).

2. Kaggle public leaderboard score (and submission settings).



3. Brief qualitative examples or error analysis highlights (optional).

6 Discussion

1. What worked best and why.

The SigLIP worked best because it is a model that classified very successfully in its original use cases, and it is a ViT which tends to work better than CNNs for these cases.

2. Failure cases, overfitting/underfitting observations.

With most models including efficientnet and RegNet, overfitting was a big issue. High augmentation tends to help reduce this issue by artificially expanding the small training set. However, we noticed that aggressive augmentation tended to hurt performance. This is likely due to it causing ambiguity due to the reduced class size of 10 images per class. It required a balance of moderate augmentation combined with regularization techniques and label smoothing.

3. Limitations and concrete next improvements.

Limitations that affected our group were definitely the dataset size and overfitting. Outside of modeling and data, hardware limitations and computing power were an issue.

Concrete next improvements to implement are ensembling and K-fold cross validation.

7. Reproducibility

1. Random seeds and determinism settings.

We ensured that the random seed in all of the libraries are set to 42, and the num_workers is set to 0.

2. Package versions / environment setup steps.

There was no need for a virtual environment, or anything of the like because we were using google colab and could just import anything we needed manually or just run a “pip install _” for any libraries we needed.

3. Exact commands to train and to generate submission.csv

Run the notebooks from top to bottom. There is no need to run the kaggle cell, so long as you have the train and test sets saved to

“/root/.cache/kagglehub/competitions/ucsc-cse-144-spring-2026-final-project” which is saved in the path variable. The kaggle cell will also only work with your own token. For security reasons we have deactivated our own.

8 Team Contributions

1. Member 1: Sarah Reuter

I wrote the data pipeline, and then worked on EfficientNetB0 and EfficientNetB2. After minimal success using those models, I added phase training to the Siglip model after the training loop, and to bring up the accuracy.

2. Member 2: Catherine Mikhailova

I tested working with a standard ViT model, and was able to improve accuracy up to 60% by using LoRA (Low-Rank Adaptation). I then decided to try using the SigLIP model with LoRA instead.

3. Member 3: Malco Salcedo

I fine tuned the RegNetY-8GF model and experimented heavily with selective unfreezing and different learning rates. I also figured out how to improve t4 gpu efficiency by ~2x using AMP float16.

9 References

1. TorchVision documentation / pretrained model sources.

2. Any papers or tutorials used.

CutMix: <https://docs.pytorch.org/vision/main/generated/torchvision.transforms.v2.CutMix.html>

Transforms: <https://docs.pytorch.org/vision/main/transforms.html>

PreTrained Models: <https://docs.pytorch.org/vision/main/models.html>

Siglip Model: https://huggingface.co/docs/transformers/en/model_doc/siglip

Pytorch AMP: <https://docs.pytorch.org/docs/2.12/amp.html>

Pytorch DataLoader: <https://docs.pytorch.org/docs/2.12/data.html>